



Assignment – 1

Submitted to:

Mobaswirul Islam
Lecturer
Department Of Computer Science & Engineering

Submitted by:

Group Name	Spectrum
Members Name	ID
Md. Golam Mahmud (Nafiz)	011201333
Anika Tabassum Orchi	011201346
Anindita Chakma	011201284
Sabrina Zaman	011212137
Safrin Sanzida Islam	011212124
Section: B Course: Software Testing and Quality Assurance/Software Testing, Verification and Quality Assurance Course Code: CSE 4495/CSE 495	

Date of Submission: September 1, 2025

➤ Test Plan

Name	Endpoint	Input / Parameter Choices	Expected Output
Authentication API	POST /auth/register	Valid Body: Contains unique username, unique email, and password.	201 Created: Returns the new user object (without the password).
		Duplicate Data: Body contains an email or username that already exists.	409 Conflict: Returns an error message like {"error": "Email already in use"}.
		Invalid/Missing Data: Body has an invalid email format or is missing a required field.	400 Bad Request: Returns a validation error message.
	POST /auth/login	Valid Credentials: Body contains the correct email and password for an existing user.	200 OK: Returns a JWT, e.g., {"token": "<JWT>"}
		Invalid Credentials: Body contains an incorrect password or a non-existent email.	401 Unauthorized: Returns a generic error like {"error": "Invalid credentials"}.
	GET /auth/profile	Valid Token: Authorization header contains a valid, non-expired JWT.	200 OK: Returns the profile data for the authenticated user.
Invalid/Missing Token: Authorization header is missing, malformed, or has an expired token.		401 Unauthorized: Returns an authentication error.	
Users API	GET /users	No Parameters: No query parameters are provided.	200 OK: Returns an array of user objects with default sorting.
		Valid Sorting: Query parameters sortBy and order are valid	200 OK: Returns a sorted array of user objects.
	GET /users/:id	Existing User ID: The :id in the path corresponds to an existing user.	200 OK: Returns the specified user's public data.
		Non-existent User ID: The :id in the path does not exist.	404 Not Found: Returns a "not found" error.
	PUT /users/:id	Valid Owner: Authorization token subject matches :id. The request body is valid.	200 OK: Returns the updated user object.
		Non-Owner: Authorization token subject does not match :id.	403 Forbidden: Returns an access denied error.
		No Token: Authorization header is missing.	401 Unauthorized: Returns an authentication error.
	GET /users/:id/contests	Existing User ID: The :id in the path exists.	200 OK: Returns an array of contests the user has joined.
		Non-existent User ID: The :id in the path does not exist.	404 Not Found: Returns a "not found" error.
	GET /users/:id/submissions	Existing User ID: The :id in the path exists.	200 OK: Returns an array of submissions made by the user.

	GET /users/:id/rating	Existing User ID: The :id in the path exists.	200 OK: Returns the user's rating information.
Contests API	POST /contests	Valid Token & Body: User is authenticated and the request body contains all required fields.	201 Created: Returns the newly created contest object.
		Missing Fields: Request body is missing required information like name or startTime.	400 Bad Request: Returns a validation error.
	GET /contests/:id/problems	Active/Past Contest: The :id corresponds to a contest whose startTime is in the past.	200 OK: Returns an array of the contest's problem objects.
		Upcoming Contest: The :id corresponds to a contest whose startTime is in the future.	403 Forbidden: Returns an error indicating problems are not yet visible.
	POST /contests/:id/join	Valid Join: Authenticated user joins a contest that is not their own and has not ended.	200 OK: Returns a success confirmation.
		Join Own Contest: The authenticated user is the creator (setter) of the contest.	403 Forbidden: Returns an error like {"error": "You cannot join your own contest"}.
		Join Ended Contest: The contest's end time has already passed.	403 Forbidden: Returns an error like {"error": "Contest has already ended"}.
Problems & Submissions API	GET /problems/:id	Public Problem: The :id belongs to a problem from a past or active contest.	200 OK: Returns the problem's details.
		Private Problem: The :id belongs to a problem from an upcoming contest.	403 Forbidden: Returns an access denied error.
	POST /problems/:id/submit	Valid Submission: Authenticated user has joined the running contest for the given problem :id.	202 Accepted: Returns the new submission's ID and status.
		Contest Not Running: The submission is for a problem in a contest that has ended or not yet started.	403 Forbidden: Returns an error like {"error": "Contest is not active"}.
		User Not Joined: The authenticated user has not joined the contest.	403 Forbidden: Returns an error indicating the user is not a participant.
	GET /submissions/:id	Owner Viewing: Authorization token's user ID matches the submission's owner.	200 OK: Returns the full submission details, including source code.
		Non-Owner Viewing: Authorization token's user ID does not match the submission's owner.	403 Forbidden: Returns an access denied error.
		Non-existent Submission: The :id in the path does not exist.	404 Not Found: Returns a "not found" error.

➤ Test Cases

- ❖ For detailed test cases kindly visit the mentioned link: [Test Cases](#)

➤ Defect Reports

- ❖ For detailed defect reports kindly visit the mentioned link: [Defect Reports](#)

➤ Risk & Recommendations

○ Risks:

- **Disclosure of Information and User List:** Logged-in users appear to have public access to the **GET /users** and **GET /submissions** endpoints. The metadata is visible to anyone and for every submission on the platform, and the entire user base can be scraped via this route, which poses a privacy risk. Also, attackers can utilise this data to pinpoint particular users, locate valuable accounts, or encompass trends in platform activities.
- **Lack of Account Lockout and Rate Limiting:** No defence against brute-force attacks is mentioned in the specification. Password guessing and credential stuffing are threats to the **POST /auth/login** endpoint. In a similar vein, mass spam account creation could be accomplished by abusing the **POST /auth/register** endpoint. This weakness can be a gateway to DOS attacks or account takeovers.
- **Insecure Direct Object References and Broken Access Control:** Some endpoints, like **PUT /users/:id**, accurately identify ownership checks, but others are hazardingly obscure. Since there is no precise rule for the **PUT /problems/:id** endpoint to be authorised, ill users may be able to change any issue on the platform. Attackers can easily locate legitimate IDs to test for these vulnerabilities thanks to public listing endpoints.
- **Poor JWT Administration:** The system lacks a token revocation method, allowing the attacker with a stolen JWT to remain valid for an hour and exploit the system.
- **Resource constraints and a lack of input validation:** Validation of the size or content of submissions (**POST /problems/:id/submit**) or problem descriptions (**POST /contests/problems**) is not mentioned. This could enable an attacker to insert malicious scripts into problem statements that are subsequently viewed by other users, or initiate DOS attack.

○ Recommendations:

- **Implement the principles for least data exposure:** The Least Privilege Principle can be applied to data exposure by restricting admin-only endpoints and limiting visible submission lists, thereby protecting user privacy.
 - **Secure authentication and registration flows:** To prevent spam, credential stuffing, and automated brute-force attacks, enforce stringent rate limiting, account lockout procedures, password complexity requirements, and email verification for safe registration and authentication.
 - **Enhance JWT and session management:** Implement a token blocklist in Redis to manage sessions and JWT, ensuring unique identifiers are added to the list when users log out or change passwords.
 - **Enforce strict input validation and resource constraints:** Implement tight resource limits and input validation to protect against resource exhaustion and injection attacks by limiting submission size, sanitizing input, and ensuring appropriate data types.
 - **Default to deny on all access control:** For PUT /problems/:id, the rule must be that only the original contest setter or an admin can update." Assume no access unless a rule explicitly grants it.
-